LLM-Based PLC ST Generation and Verification

GenAI Group 19

GenAI for Industrial Programming

[University, City, Postcode]

[Tel]

[Email]

Abstract

Large language models can draft Programmable Logic Controller (PLC) programs, but compilable Structured Text (ST) does not establish correct behaviour at each scan cycle. This report documents a 34-task benchmark and a generation harness that combines task cards, candidate generation, fast and authoritative compilation, per-cycle functional tests, bounded repair, and explicit termination. The tasks combine 20 public problems with 14 adapted or synthesised problems and contain 316 I/O points, 144 constraints, 177 Linear Temporal Logic (LTL) clauses, and 214 test cases. We evaluated the baseline (Aider 0.86.2) and our harness with two language models in four settings covering 136 final runs. Full-task pass counts were E1 18/34, E2 16/34, E3 8/34, and E4 7/34. On this task set, in this configuration, in this single run, compile rates exceeded full-task pass rates in every setting, and the baseline settings produced more full-task passes than our harness. The result cannot support a general ranking because each task ran once, E3 had 24 budget terminations influenced by a token-counting defect, five task copies differed from the frozen test set, three tasks had disputed criteria, and formal verification was skipped for all 136 runs.

Keywords: industrial programming; PLC; Structured Text; linear temporal logic; per-cycle verification

1 Introduction

Programmable Logic Controller (PLC) software governs machines that repeat a scan cycle thousands of times per shift. Structured Text (ST) is one of the languages defined by IEC 61131-3, and industrial implementations must also respect safety requirements and signal conventions beyond syntax. [1], [2], [3]

Linear Temporal Logic (LTL) gives the evaluation a way to express properties such as stopping on the next scan, holding an output until a completion signal, or keeping two actuators mutually exclusive. The repository under study combines these properties with cycle-level test cases, so a program can compile yet still fail the functional criterion. [4]

This report asks whether an LLM agent can move from a natural-language task card to an ST program that passes both the authoritative compiler and the frozen per-cycle suite. It compares our harness with the baseline (Aider 0.86.2) under two model configurations. The study also records where the evidence stops: one run per task, mismatched test copies, disputed criteria, incomplete token accounting, and no completed formal verification.

The work contributes a 34-task specification set, a seven-module generation harness, a 136-run result matrix that can be recomputed from raw records, and an explicit account of validity threats. The conclusion is deliberately narrow. It describes the observed engineering process on this task set, in this configuration, in this single run; it is not a leaderboard for models or agent architectures.

2 Experimental Environment

2.1 Systems and Toolchain

The authoritative compiler was CODESYS x64 3.5.22.40 with the CODESYS V3.5 SP22 Patch 4 profile. The target was CODESYS Control Win V3 x64 with a 20 ms task. MATIEC served as the fast syntax and type check. A CODESYS result, rather than the fast check, determined whether a candidate entered the functional tests. [5], [6]

Table 1 records the software and model configuration that the final matrix used.

Table 1. Software, model, and tool versions used in the frozen experiment

Item	Configuration
Operating system	Windows 64-bit, build 10.0.26200
Python	Core supports 3.10 or later; validation records 3.12.14
CODESYS	3.5.22.40; V3.5 SP22 Patch 4
CODESYS scripting	4.2.0.0 with IronPython 2.7
Target device	CODESYS Control Win V3 x64
Core libraries	Standard 3.5.22.0; IoStandard 3.5.22.0
Baseline	Aider 0.86.2
Models	deepseek-v4.1-flash; gpt-6.1-sol
Model access	DMXAPI Chat Completions with model and messages
Scan cycle	20 ms

The transfer record did not include CPU, memory, or storage identifiers. We therefore report the software configuration but do not invent hardware specifications. All frozen runs used one machine, and a complete cross-machine rerun of the model, compiler, and test executor remains unverified.

2.2 Access Methods and Resource Limits

The two models were reached through a Chat Completions compatible endpoint. The API key came from an environment variable and did not enter the source tree or result logs. Aider used its own OpenAI-compatible provider path but shared the task input, authoritative compiler, functional tests, and external timeout contract.

Table 2 lists the resource limits that constrained this contract.

Table 2. Execution limits applied to both frameworks

Limit	Value	Operational role
Single-task hard timeout	900 s	Terminates the task process tree
Authoritative compile timeout	90 s	Bounds CODESYS script compilation
Fast-check target	3 s	Returns early syntax and type feedback
Context budget preset	6000 token	Local estimate, not provider billing
Cumulative token threshold	100000	Local stop rule for our harness
Candidates	3 default; 5 maximum	Candidate diversity before selection
Repair rounds	5 maximum; stop after 2 non-improving rounds	Bounded repair loop
Concurrency	2 slots	Limits simultaneous task processes
Random controls	Seed 42 local identifier; not sent by default	Reproduction label, not a server guarantee

The environment check exercised 6 diagnostic probes and 34 declaration skeletons. A total of 33 skeletons generated target code; P18 failed because the current profile does not support VARIANT. Runtime materials report 121.4 s for the complete IDE startup and skeleton check, 33.8 s for the correct-example dual channel, and 32.2 s for the error-example channel. These are environment checks, not model pass rates.

The 6000-token context value and the 100000-token threshold were local control values. The confirmed accounting evidence came from provider usage fields when available. Missing usage remained unknown; it was not replaced by an estimate.

3 Dataset and Formal Specifications

3.1 Dataset Composition

The frozen set contains 34 tasks: 20 public tasks P01-P20 and 14 self-built tasks S01-S14. Each task has a structured task card, an I/O table, an initial state, numbered constraints, error examples, and a linked specification file. The complete set contains 316 I/O points, 144 constraints, and 177 LTL clauses.

Table 3 summarises the resulting dataset and test-suite scale.

Table 3. Dataset and test-suite scale

Measure	Value
Tasks	34: 20 public and 14 self-built
I/O points	316
Numbered constraints	144
LTL clauses	177
Difficulty	9 easy, 14 medium, 11 hard
Scenario categories	6 A, 11 B, 11 C, 6 D
Functional test cases	214
CSV rows and scan steps	22636 rows and 22422 executed scan steps
Case categories	68 normal, 38 boundary, 48 abnormal, 60 safety

3.2 Construction and Adaptation

The public tasks were parsed from the course bank by an import script. Editors added initial-state details, K identifiers, error examples, and LTL links without replacing the source interface. The 14 self-built tasks comprise 8 adaptations and 6 tasks written from an industrial scenario. Each adaptation records what changed and why in the provenance table.

Self-built tasks followed a fixed sequence: select a scene that can be judged within one page, write incorrect implementations before writing the requirement, derive numbered K constraints from those failures, define a compact I/O interface, select clauses from the clause library, and perform a second-person review. This sequence forced authors to name a detectable failure before adding descriptive text.

The public description applies to the initial import. Seven public tasks and S11 were later revised before the final matrix. The source table and final task files preserve both stages, and the report treats the final hashes as the experimental input rather than claiming that the initial wording remained unchanged.

3.3 Test-Case Schema and Formal Specifications

The instructor case template requires a case number, purpose, preconditions, input sequence, expected result, and coverage. The repository stores the trace as one CSV row per scan cycle. Table 4 maps those required elements to the encoded fields and uses case P03-01 as an example.

Table 4. Traceability of one representative functional test case

Element	Encoded field	P03-01 example
Case number and purpose	case_id and purpose	P03-01: dual-side hysteresis
Precondition	cycle=0 row	Value 0; thresholds 10/3; no alarm
Input sequence	Input columns for cycle 1 or later	12, 9, 7, 2, 6
Expected result	Output columns for each cycle	High alarm, hold, clear, low alarm, clear
Coverage	spec_refs	K3; K4; C7; C7b; C10
Category	category	Normal
Execution	runner and case result	CODESYS artifact plus per-cycle comparator

The formal layer uses a clause library C1-C10, covering stop priority, start permission, state retention, output exclusion, response time, alarm latching, hysteresis, count monotonicity, timed lower bounds, and initial state. Time ranges are expressed in scan cycles; for example, the property G(hiAlarm -> (hiAlarm W Value < hiLevel - hystWindow)) keeps a high alarm until the value enters the hysteresis band. G denotes a condition that holds on every suffix; W denotes weak until.

3.4 Validation of Data and Specifications

The LTL parser read all 34 specification files and all 177 clauses, and traced each atomic proposition to a task I/O name or a declared alias. The test directory holds 214 cases, 22636 CSV rows, and 22422 executable scan steps. Every encoded K key has at least one case reference. These checks establish structural linkage, not semantic proof: the run records mark formal verification as skipped for all 136 final runs.

The case suite includes a frozen and an experiment copy. Five task copies later differed: P01, P08, P17, P18, and P19. Four differences did not change the observed pass decision in the affected runs. P19 used an older copy with reversed emergency stop polarity. Section 6 treats this as a validity issue rather than correcting the matrix after the fact.

4 Framework Design

4.1 Architecture

Our harness separates task organisation, generation, evaluation, repair, and logging. The seven implementation modules are parse, task_card, generate, tools, repair, terminate, and logging. Figure 1 shows the two agent paths entering one shared toolchain. The baseline feeds its edited program into that toolchain; the self path also uses candidate selection and the bounded repair loop.

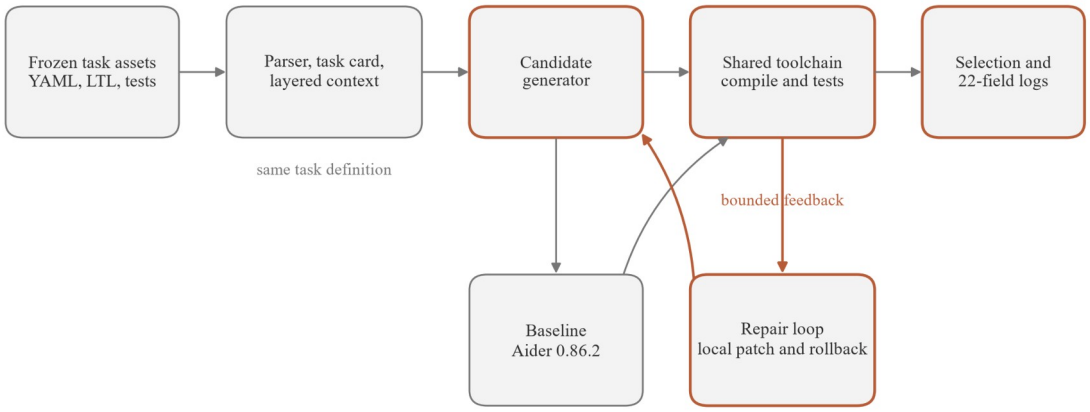


Figure 1. Architecture of our harness, the baseline (Aider 0.86.2), and the shared evaluation path.

The task-card layer preserves the original YAML, projects a declaration interface, and records semantic warnings. It does not silently fill missing requirements. The context builder keeps the task, I/O, constraints, and LTL as mandatory content; it removes optional references first and records the removal decision.

The tools module normalises diagnostics to code, severity, line, message, and raw text. It calls the fast checker, the CODESYS bridge, and the per-cycle test bridge. The terminal record keeps the 22 required result fields while process events and candidate evaluations remain in sidecar files.

4.2 Execution and Termination

A run starts by computing its task-set fingerprint and checking for an existing terminal record. New work acquires a per-task lock, builds the context, generates candidates, and evaluates each candidate. A compile failure can be sent to the repair loop; a compiled candidate proceeds to every valid case for the task. A repair patch must preserve the candidate's passing case set or the harness rolls it back.

Figure 2 shows the resulting state machine and its bounded feedback edge.

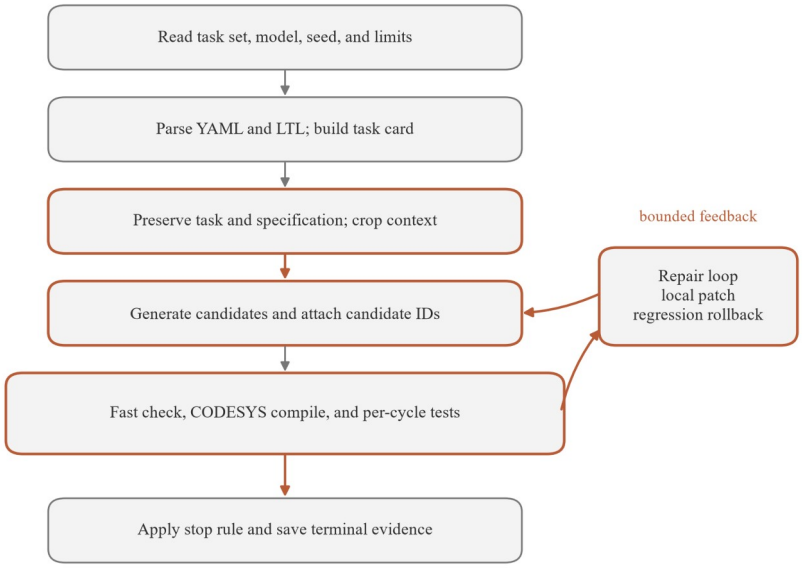


Figure 2. Execution flow for one task, including candidate evaluation, the repair loop, and the terminal stop rule.

Termination combines a compile state, the full case result, a repair limit, elapsed time, and the local token threshold. The maximum repair count was five, with an early stop after two rounds without improvement. The parent process enforced the 900 s limit and retained the latest evidence before moving to the next task.

Budget termination was a first-class terminal state. It did not become a functional pass, and it did not become a generic model failure. This distinction matters because E3 had 24 budget terminations, including 19 runs with a compilable artifact and 5 without an accepted artifact.

4.3 Design Rationale

Table 5 records the main design decisions and the evidence that constrained them. The table includes costs as well as benefits because several choices improved traceability while adding runtime or limiting comparability.

Table 5. Major design decisions and their rationale

Decision	Alternative considered	Why this one	Cost or evidence
Use layered context with mandatory task and LTL content	Send the largest possible prompt every round	Protects requirements from silent truncation	6000-token local estimate; context_trace records removals
Generate three candidates by default	Generate one candidate per task	Tests multiple implementation strategies without an unbounded search	3 default, 5 maximum; generation cost rises with candidate count
Use fast check and CODESYS as two verification channels	Use only the fast parser	Separates early diagnostics from the authoritative compiler	33 of 34 skeletons passed CODESYS; P18 exposed a dialect limit
Filter and rank on compile state before functional score	Run every functional case first	Avoids spending test time on programs the authority cannot build	Compile passes remained higher than full-task passes in all settings
Repair through local JSON patch operations	Rewrite the complete ST file each round	Keeps the reviewed region visible and rejects ambiguous edits	E3 mean repair 0.00; E4 mean repair 1.56
Rollback a patch that loses a passing case	Accept any patch that reduces compile errors	Protects previously working behaviour	Depends on stable case IDs and complete per-cycle results
Use explicit time, budget, and stagnation stops	Use a fixed retry count alone	Represents both quality and operational limits	E3 recorded 24 budget terminations
Keep the baseline's native agent strategy	Apply our candidate and repair policy to Aider	Preserves the distinction between two agent designs	The baseline (Aider 0.86.2) retained its own context and reflection behaviour
Keep one shared result schema	Use a separate schema for each framework	Makes run-level comparison and auditing possible	136 final records share the 22-field contract
Leave unavailable formal verification as skipped	Treat a parsed LTL file as a passing proof	Prevents an unsupported verification claim	136 of 136 final runs were skipped

4.4 Baseline and Comparison Boundary

The baseline (Aider 0.86.2) received the same task, files, external timeout, CODESYS compiler, and per-cycle judge. It used its own context construction, edit format, and reflection behaviour. Applying our candidate and repair policy to Aider would have removed the design contrast, so the implementation deliberately kept the native baseline behaviour. [7]

The shared contract still leaves internal budget differences. Aider could edit before the outer framework observed the first artifact, while our harness evaluated every named candidate. Aider also did not use the same local token threshold. The comparison therefore evaluates two complete configurations, not two isolated model calls.

5 Results and Analysis

5.1 Outcome Definitions and Frozen Matrix

We define full-task pass as an authoritative compile pass plus a pass on every valid functional case for that task. Compile rate uses all 34 tasks in the denominator. The functional-case rate uses only cases that entered testing, so its denominator changes with compile and budget outcomes. These measures answer different questions.

The matrix contains 136 final runs: four settings and 34 tasks per setting. Thirty-two runs replaced earlier records after criterion revisions, and 104 original runs were retained. Replacement records replace their predecessors; they are not added to the 136-run count.

Table 6 reports the full-task pass count, compile count, and latency summary for each setting.

Table 6. Core results for the four frozen settings

Setting	Framework and model	Full-task pass	Compile pass	Median / p95 latency
E1	baseline / deepseek-v4.1-flash	18/34 (52.94%)	32/34	553.0 s / 830.4 s
E2	baseline / gpt-6.1-sol	16/34 (47.06%)	32/34	299.7 s / 594.7 s
E3	self / deepseek-v4.1-flash	8/34 (23.53%)	28/34	357.1 s / 540.5 s
E4	self / gpt-6.1-sol	7/34 (20.59%)	33/34	268.8 s / 636.4 s

5.2 Four-Setting Comparison

On this task set, in this configuration, in this single run, E1 produced 18 full-task passes, E2 produced 16, E3 produced 8, and E4 produced 7. On this task set, in this configuration, in this single run, the two baseline settings therefore produced more full-task passes than the two harness settings.

On this task set, in this configuration, in this single run, compile rates were 94.12% for E1, 94.12% for E2, 82.35% for E3, and 97.06% for E4. On this task set, in this configuration, in this single run, every compile rate exceeded the corresponding full-task pass rate; the gap shows that a successful build did not establish functional correctness.

Figure 3 plots the two pass definitions side by side.

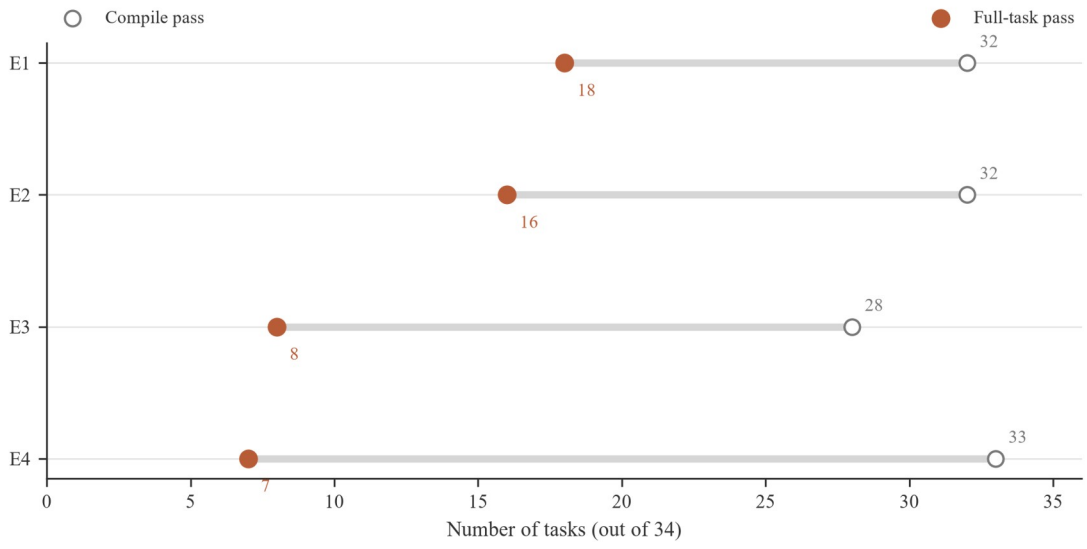


Figure 3. Compile passes and full-task passes for the four settings; each setting covers all 34 tasks.

On this task set, in this configuration, in this single run, E4 compiled 33 of 34 tasks but produced 7 full-task passes, whereas E1 compiled 32 and produced 18. On this task set, in this configuration, in this single run, the compile count alone would reverse the ordering that the full-task result shows.

5.3 Difficulty and Paired Outcomes

Table 7 separates full-task passes by the 9 easy, 14 medium, and 11 hard tasks.

Table 7. Full-task passes by difficulty

Setting	Easy (9)	Medium (14)	Hard (11)
E1	6/9	8/14	4/11
E2	7/9	6/14	3/11
E3	5/9	2/14	1/11
E4	4/9	2/14	1/11

Figure 4 shows the same counts as within-level pass rates.

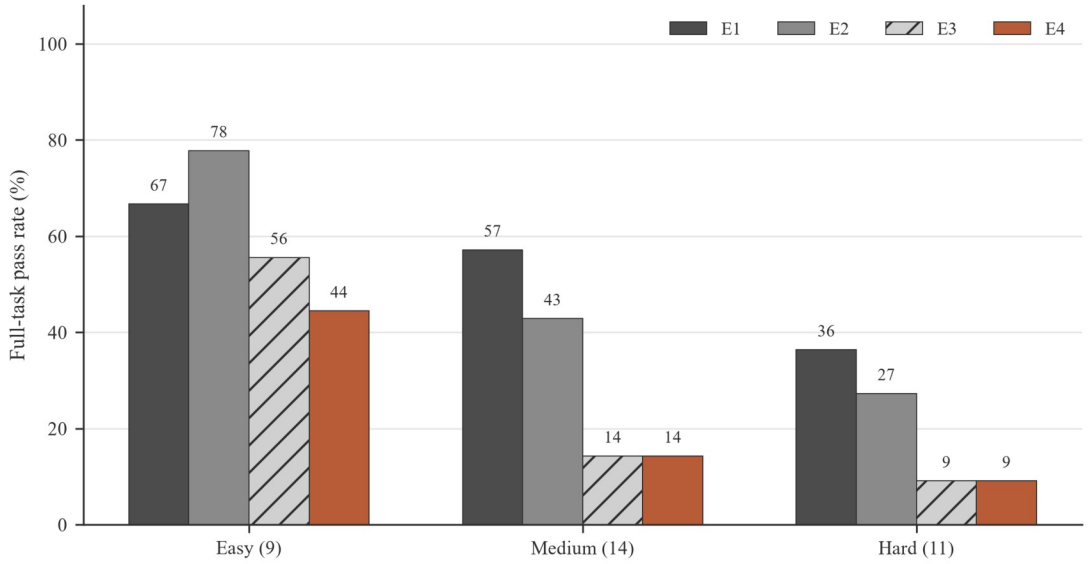


Figure 4. Full-task pass rate by task difficulty for each setting.

On this task set, in this configuration, in this single run, performance fell with difficulty in every setting. E1 passed 6 of 9 easy tasks, 8 of 14 medium tasks, and 4 of 11 hard tasks. On this task set, in this configuration, in this single run, E3 passed 5 of 9 easy tasks, 2 of 14 medium tasks, and 1 of 11 hard tasks.

For DeepSeek, 8 tasks passed in both E1 and E3, 10 passed only in E1, and 0 passed only in E3. For GPT, 7 tasks passed in both E2 and E4, 9 passed only in E2, and 0 passed only in E4. On this task set, in this configuration, in this single run, the paired overlap is consistent with the aggregate counts but still describes only one sample per task.

Within Aider, 15 tasks passed under both models, 3 passed only with DeepSeek, and 1 passed only with GPT. Within our harness, 6 tasks passed under both models, 2 passed only with DeepSeek, and 1 passed only with GPT. On this task set, in this configuration, in this single run, the model difference was smaller than the framework difference.

5.4 Failure Attribution

E3 recorded 24 budget terminations, of which 19 had a compilable artifact and 5 had none, plus 2 timeouts. Its mean repair count was 0.00. The zero value does not show that repair was unnecessary;

the budget stop often arrived before a repair round. E4 reached 25 functional failures, 1 compile failure, and 1 timeout, with a mean repair count of 1.56.

E1 had 13 functional failures, 2 timeouts, and 1 tool error. E2 had 16 functional failures, 1 compile failure, and 1 timeout. These terminal categories separate model or program failure from an unavailable tool or an expired task.

On this task set, in this configuration, in this single run, the token budget check in the original harness used UTF-8 byte length as a stand-in for token count against a 100000-token threshold. The implementation could therefore count non-ASCII text as more tokens than the provider would. The matrix keeps that behavior because the submission had already run; changing the harness and rerunning would create a new experiment rather than repair the old one.

5.5 Resources and Independent Replay

Table 8 reports known token use and latency; Figure 5 displays the known token totals.

Table 8. Known token usage and run-time summaries

Setting	Known input	Known output	Unknown runs	Median / p95
E1	537,229	987,897	8	553.0 / 830.4 s
E2	1,026,727	180,806	1	299.7 / 594.7 s
E3	103,954	943,390	0	357.1 / 540.5 s
E4	211,195	121,692	1	268.8 / 636.4 s

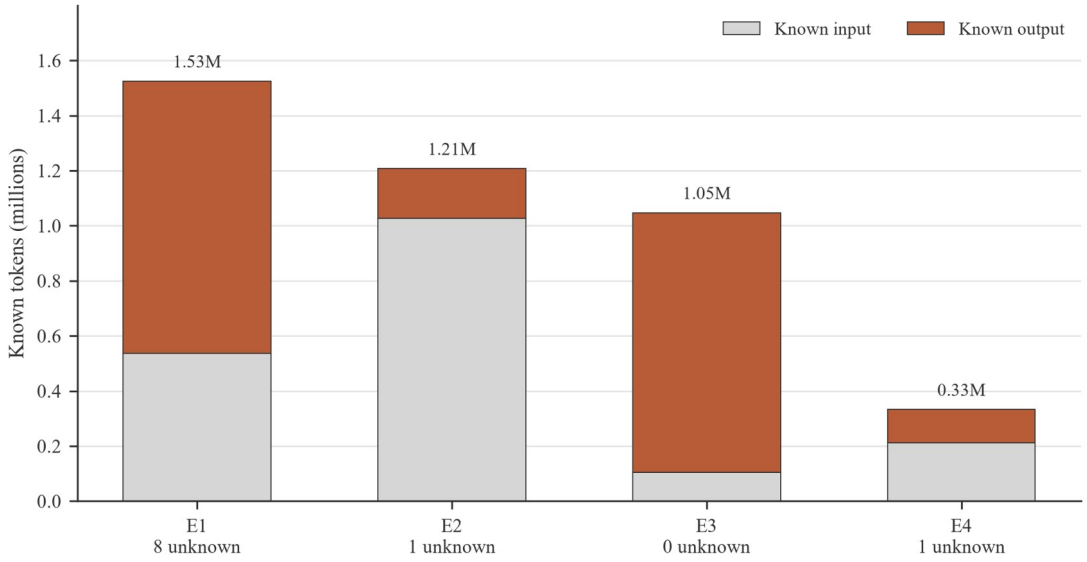


Figure 5. Known token use by setting, with the number of runs whose provider usage was unavailable.

Unknown usage appeared in 10 of 136 runs: 8 in E1, 1 in E2, 0 in E3, and 1 in E4. On this task set, in this configuration, in this single run, the known totals were 1.53 million tokens for E1, 1.21 million for E2, 1.05 million for E3, and 0.33 million for E4. These totals are lower bounds for the observed resource use and do not form a price comparison.

An independent interpreter replay processed 132 runs with source. It completed 72, reported 60 unsupported-dialect errors, and could not start 4 abandoned runs without source. The replay is an audit of saved ST files. It does not replace the CODESYS actual traces, and its failures do not enter the official pass counts.

A reader who stops after this section should carry four limits into the conclusion. Each task ran once, E3's mean repair count was 0.00 under a defective token-budget check, five task copies differed from the frozen set and three disputed tasks remained unresolved, and all 136 run records marked formal verification as skipped.

6 Threats to Validity

6.1 Single Run and Internal Validity

Each task ran once per setting. We did not estimate run-to-run variance, model sampling variance, or gateway variance. Seed 42 was a local run identifier and was not sent to the model API by default. The observed count difference between settings cannot be separated from model updates, service-side scheduling, or internal baseline edits that occurred before the outer evaluation.

The baseline and our harness also used different internal control policies. The baseline (Aider 0.86.2) retained its own reflection and edit loop, while our harness used named candidates, local patches, and an explicit token threshold. A result on this task set, in this configuration, in this single run identifies a difference between complete configurations; it does not isolate a single algorithm.

6.2 Budget Accounting and Case Freeze

The original budget check counted UTF-8 bytes as tokens. This defect may have ended E3 repair earlier than the stated 100000-token threshold intended. Ten of 136 runs also lacked provider usage data, so the reported token totals are incomplete. The combination weakens any equal-budget interpretation.

Five tasks used experiment test-case copies that differed from the frozen suite: P01, P08, P17, P18, and P19. The first four differences did not alter the affected pass decision. P19 used an older emergency-stop polarity. The experiment still used one copy for all four settings, so the internal comparison retained a common criterion, but that criterion was not the final frozen copy.

Three disputed tasks were P19, S11, and S05-06. P19 mixed a criterion version with real emergency-stop behaviour; S11 had an undefined transition and a blink-phase boundary; S05-06 expected a second increment without a second rising edge. The matrix retains the submitted decisions. Correcting these cases and rerunning would be a new study with new numbers.

6.3 Formal Verification and Construct Validity

All 136 final records marked formal verification as skipped. The 177 clauses passed parsing and proposition mapping, but no model checker established that a generated ST program satisfied them. Functional pass therefore means agreement with the finite case set, not a proof over all reachable states.

The test suite is broad enough to expose timing and safety failures: 214 cases cover normal, boundary, abnormal, and safety classes, with 22422 executed scan steps. It still cannot exhaust every input sequence for a stateful controller. Some constraints, including exact error-code semantics and array updates, also depend on case-level checks rather than a compact LTL formula.

The independent replay completed 72 of 132 runs with source but hit 60 interpreter errors, mostly because the saved programs used ST features outside the interpreter subset. This audit confirms that many artifacts are inspectable, not that the official CODESYS traces can be reconstructed from the repository alone.

6.4 External Validity

The tasks fit a course-scale benchmark. They cover industrial patterns, but they do not reproduce a plant's device libraries, cycle jitter, firmware revisions, or site acceptance tests. CODESYS and

MATIEC were fixed for the run, and the full model, compiler, and test path has not been rerun on a second machine.

Model identifiers may map to moving provider versions, and the gateway may route requests differently over time. The result therefore describes the frozen records, not a stable property of the named models. Replication should preserve model version metadata, gateway settings, compiler binaries, test hashes, and complete usage fields.

7 Conclusion

The project produced a PLC ST benchmark, formal clause set, per-cycle test suite, generation harness, and reproducible 136-run result matrix. The compiler and functional results measure different levels of correctness. In this run, compile rates were high while full-task passes remained lower, so a successful build was insufficient evidence of scan-cycle behaviour.

On this task set, in this configuration, in this single run, the baseline settings produced more full-task passes than our harness. The comparison cannot become a general claim because the study used one run per task, a defective token-budget check, five mismatched case copies, three disputed tasks, and no completed formal verification.

The next experiment should freeze one case set, fix tokenizer accounting, repeat every setting, preserve complete per-cycle traces, and run the model checker. Those changes should create a new batch with new identifiers. They should not overwrite the results used in this report.

References

[1] International Electrotechnical Commission, Programmable controllers - Part 3: Programming languages, IEC 61131-3:2013, Geneva, Switzerland, 2013.

[2] International Organization for Standardization, Hydraulic fluid power - General rules and safety requirements for systems and their components, ISO 4413:2010, Geneva, Switzerland, 2010.

[3] NAMUR, Signal level for fault information from measuring transducers, NE 43:2003, Leverkusen, Germany, 2003.

[4] Zertz777, Gen-AI: PLC ST generation and cycle-level verification repository, 2026. [Online]. Available: https://github.com/zertz777/Gen-AI

[5] CODESYS GmbH, CODESYS Development System V3.5 SP22 documentation. [Online]. Available: https://content.helpme-codesys.com/

[6] Beremiz Project, MATIEC compiler source and command-line documentation. [Online]. Available: https://github.com/beremiz/matiec

[7] Aider AI, Aider documentation. [Online]. Available: https://aider.chat/docs/

Appendix A Per-Task Specification Summary

Table 9 and Table 10 summarise the frozen task set used in the experiment. LTL counts come from the specification files. K coverage compares the distinct K identifiers in the task card with those referenced by test cases. Full formulas remain in the repository.

Table 9. Specification summary for public tasks P01-P20

Task	Difficulty	Category	I/O	LTL	K coverage
P01	Easy	C	11	6	4/4
P02	Easy	B	8	5	4/4
P03	Easy	D	8	6	4/4
P04	Easy	D	7	5	5/5
P05	Easy	A	5	5	4/4
P06	Easy	A	4	5	4/4
P07	Medium	A	7	5	4/4
P08	Medium	B	10	5	5/5
P09	Medium	B	9	5	4/4
P10	Medium	B	7	6	4/4
P11	Medium	C	12	5	4/4
P12	Medium	D	9	5	4/4
P13	Medium	B	5	5	4/4
P14	Medium	B	2	2	4/4
P15	Hard	C	16	5	4/4
P16	Hard	C	13	5	4/4
P17	Hard	C	16	5	4/4
P18	Hard	B	7	5	4/4
P19	Hard	C	24	6	5/5
P20	Hard	C	17	5	4/4

Table 10. Specification summary for self-built tasks S01-S14

Task	Difficulty	Category	I/O	LTL	K coverage
S01	Easy	A	8	5	4/4
S02	Easy	A	8	6	4/4
S03	Medium	A	9	6	4/4
S04	Medium	B	7	5	4/4
S05	Hard	B	8	6	4/4
S06	Medium	B	9	6	4/4
S07	Hard	C	12	6	5/5
S08	Hard	C	9	5	4/4
S09	Medium	C	9	5	5/5
S10	Easy	D	6	5	4/4
S11	Hard	D	7	4	4/5
S12	Medium	D	8	5	4/4
S13	Hard	C	11	6	5/5
S14	Medium	B	8	6	5/5